Caption-Image Retrieval

Searching an image collection with natural language

Vishanth (PES1UG24AM323) · **Kshitij Gupta** (PES1UG24AM915)

UE24CS352A Machine Learning — Mini-Project · October 2026

1. Motivation

- Enormous numbers of images are produced and stored every day
- Finding *the right* image by scrolling is painful
- What if we could search images the way we search text?

> *Type a sentence → get the images that match it.*

2. Problem Statement

Input

- A collection of raw images
- A natural-language query describing a desired image

Output

- A ranked list of the **top-K** most relevant images

Framing

- Learn a **shared embedding space** where captions and images are aligned
- Retrieval = nearest-neighbour search in that space
- Scored by **Recall@K**: % of queries whose true image is in the top K

3. Dataset — Microsoft COCO 2017

Split	Source	Images	Captions	Words / caption (mean · median)
Train	`train2017`	113,287	566,435	10.5 · 10
Validation	`train2017`, held out	5,000	25,000	10.5 · 10
Test	`val2017`	5,000	25,000	10.5 · 10

- Raw images + captions from **cocodataset.org** — first **5** captions per image
- Official test captions are private → `val2017` is our **test** set
- **Text:** GloVe 300-d word vectors — cover **99.87%** of caption tokens
- Captions are 5–50 words; **95%** have ≤ 15 words

We train on the full training split and report on the 5,000 test images (and the standard 1K protocol).

4. Preprocessing & vocabulary

- Lowercase, strip punctuation, whitespace tokenisation — same for both models
- Vocabulary built from **training captions only**
  - **26,351** word types (+ `<pad>`, `<unk>`), 10,417 seen only once
  - **82.9%** of them initialised from GloVe; the rest random, all fine-tuned
- Captions padded / truncated to **20** tokens — cuts only **0.6%** of captions
- Unseen test words → `<unk>`: **0.2%** of tokens

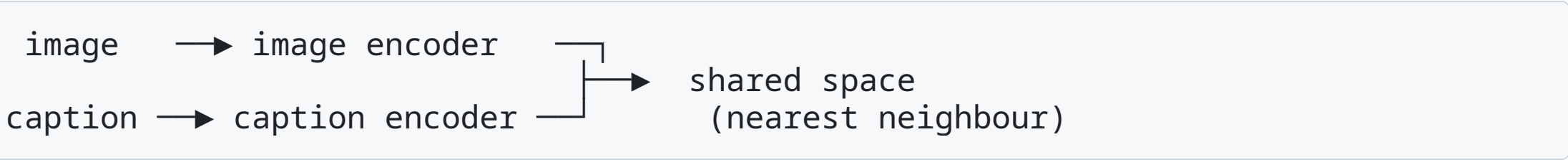
5. Image features — computed from raw JPEGs

- **VGG-19** (ImageNet) → `fc7` after ReLU → **4,096-d** per image
- Resize short side to 256, **centre crop** 224 × 224
- **L2-normalise** each feature vector — essential:

3,000-image pilot	Val R@10 (1K pool)
raw `fc7` (norms 19–82)	2% — GRU memorises, stays at chance
L2-normalised	**49%**

- `train2017.zip` is **19.3 GB** → streamed with HTTP range requests; only the 1.9 GB of features is kept
- ~123k images on an Apple **M3 Pro** GPU (PyTorch MPS)

6. Approach — a shared space



- Similar pairs → **close** in the space
- Similarity = **negative squared Euclidean distance**
- Retrieval: embed the query, return the nearest image embeddings
- Evaluation: for each of the **25,000** test captions, rank all **5,000** test images

7. Baseline — multi-response linear regression

- Caption = **mean of its GloVe word vectors** (300-d)
- Fit a weight matrix `W` so that `X @ W ≈ Y`
  - `X` = caption features (566,435 × 300), `Y` = VGG-19 image features (566,435 × 4,096)
  - each image feature repeated **5×** — one row per caption; solved from per-image sums, so the 9.3 GB `Y` is never built
- Least squares — **closed form**, no gradient descent; `W` is 300 × 4,096 (**1.2 M** weights)
- Query → `x @ W` → nearest image in VGG space

Weakness: mean-pooling **discards word order** and dilutes the object being described.

8. Model — GRU + GloVe

- **Image encoder:** learned linear map `f_i(i) = W_i · f_VGG(i)`, 4,096 → 1,024
- **Caption encoder:** GRU over GloVe-initialized embeddings, `f_c(c) = GRU(c)`
  - embedding 26,353 × 300 → GRU (hidden 1,024, final state) → linear 1,024 → 1,024
- Embeddings **L2-normalized**

Component	Parameters
Image projection	4.20 M
GRU	4.07 M
Word embeddings	7.91 M
Caption projection	1.05 M
Total	**17.2 M**

Following Vendrov et al. (2015), "Order-Embeddings of Images and Language".

9. Training objective & setup

Margin ranking loss — the correct pair must beat every wrong pair by a margin α:

```
L = Σ   Σ_{c'≠c} max(0, α − s(i,c) + s(i,c'))   +   Σ_{i'≠i} max(0, α − s(i,c) + s(i',c))
    (i,c)          wrong captions for image i              wrong images for caption c
```

- Negatives come **free from the same batch**: 127 per positive, in each direction
- Each epoch pairs every image with **one random caption** of its five

Embed / hidden	Batch	Optimizer	Margin α	Max len	Epochs
1,024 / 1,024	128 (886 / epoch)	Adam, lr 0.001	0.1	20	20

Learning rate mattered: at lr 0.05 the loss fell but Recall@K stalled near chance. Best-R@10 checkpoint is kept.

10. Implementation

- **PyTorch** — VGG-19 feature extraction, GRU + image projection, on an Apple **M3 Pro** GPU (MPS)
- **NumPy** — GloVe, closed-form baseline
- `src/` modules:
  - `remote_zip.py` `extract_features.py` `prepare_coco2017.py` — data pipeline
  - `data.py` `glove.py` `baseline.py` `run_baseline.py`
  - `vocab.py` `models.py` `train_rnn.py`
  - `retrieve.py` `demo.py`
- 20 epochs × 886 steps, ~1.4 min / epoch; best-validation-R@10 checkpoint scored once on test
- Demo embeds the 5,000 test images **once**, then answers each query in a single nearest-neighbour pass

11. Results — test set (`val2017`)

Model	Pool	R@1	R@5	R@10	Mean rank
Random (chance)	5K	0.02	0.10	0.20	2,500.5
Baseline	5K	2.8	10.7	17.5	156.7
GRU + GloVe	5K	**10.4**	**30.6**	**43.8**	**48.1**
Baseline	1K	9.7	31.6	47.7	32.2
GRU + GloVe	1K	**26.9**	**62.4**	**77.9**	**10.4**

- GRU beats the baseline on **every metric**: 5K R@10 **×2.5**, R@1 **×3.7**
- Mean rank of the true image: **157 → 48** out of 5,000 (top **1%**)
- R@10 is **~220×** chance on the 5K pool

Recall in %. 5K: each of the 25,000 test captions ranks all 5,000 test images. 1K: five 1,000-image folds, averaged (the reference protocol).

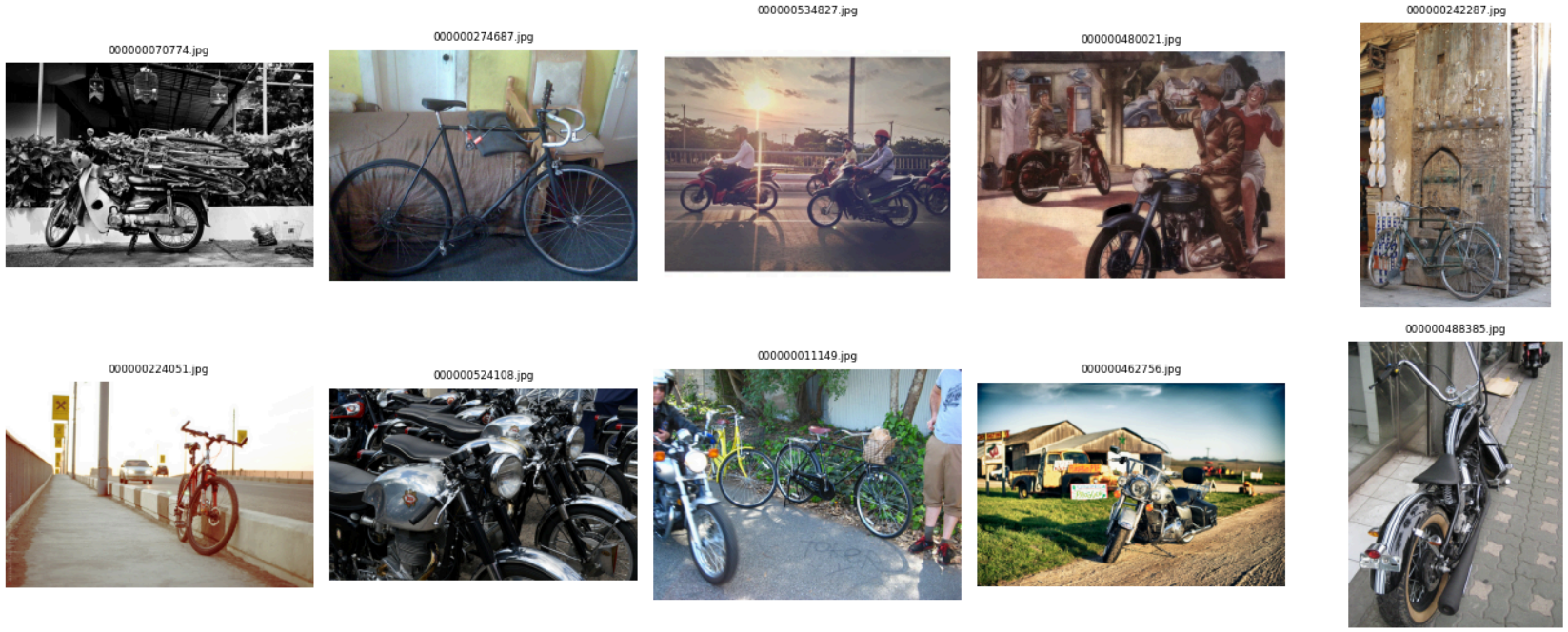
12. Full data vs our earlier subset — and the reference

	Train images	Pool	R@1	R@10	Mean rank
Earlier run (Vendrov features, val)	5,000	5K	5.0	28.3	85.6
This run (COCO 2017, test)	**113,287**	5K	**10.4**	**43.8**	**48.1**
This run (COCO 2017, test)	**113,287**	1K	**26.9**	**77.9**	**10.4**
Reference report — GRU + GloVe	113,287	1K	37.0	86.8	7.3
Vendrov et al. — GRU	113,287	1K	36.3	85.8	7.6

- **23× more training data → +15.5 points R@10** with the same model
- Remaining gap to the reference (77.9 vs 86.8): **20 vs up to 50 epochs** (still improving), **centre crop vs 10-crop** features

Reference report: Qian & Lamberti, "Neural Caption-Image Retrieval" (Stanford CS229), docs/report.pdf. Test images differ (val2017 vs Karpathy test).

13. Demo — query: "a cycle"



- **10 of 10** results are **bicycles / motorcycles** — all unseen test images
- "cycle" appears only **219 times** in training captions (vs 6,940 × "motorcycle")

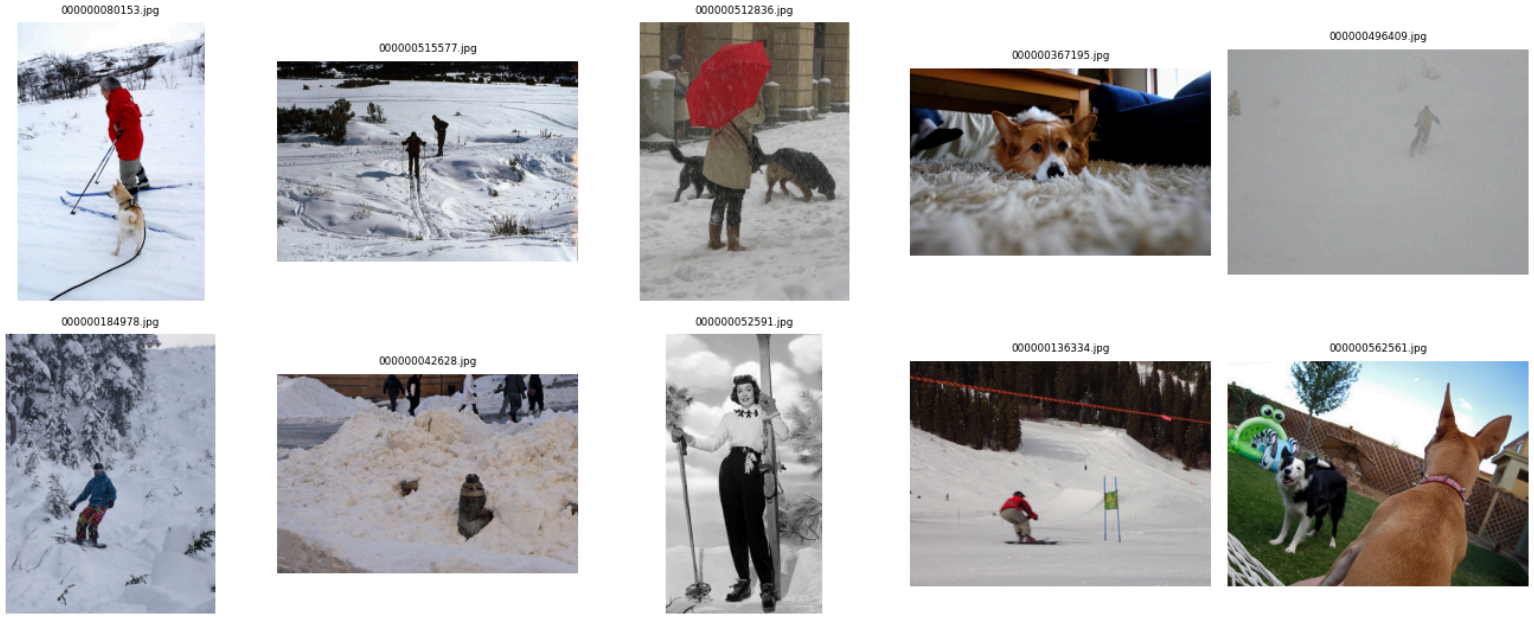
Top hit: "small motorcycle with several bicycles tied to the back of it". Our earlier 5k-image model managed 8 of 10, with a locomotive and a clock among the misses.

14. Training curve — validation, 5K pool

Epoch	1	2	5	10	15	20
Loss	180.1	90.0	58.0	40.2	29.6	23.8
R@1	4.1	4.8	6.6	8.2	9.7	10.2
R@10	23.2	27.2	33.3	38.4	42.2	**43.8**
Mean rank	89.5	74.4	63.0	54.3	48.0	**45.8**

- After **one** epoch on the full data the GRU already beats the baseline (23.2 vs 17.5 R@10)
- Still improving at epoch 20 → more epochs should help
- ~**1.4 min / epoch** on an Apple M3 Pro GPU (886 steps of 128)

15. Honest failures — "two dogs playing in the snow"



- Gets **snow** and **dogs** — but only **one** image shows *two* dogs in snow
- **Why:** one **global** VGG vector per image — no regions, no counting; the GRU sees "dogs" and "snow" but the image side cannot check they co-occur

16. Conclusions & future work

Conclusions

- Trained on the **full COCO 2017** split, with image features computed from raw JPEGs
- GRU **R@10 43.8%** (5K) / **77.9%** (1K), mean rank **48** of 5,000 — **2.5×** the baseline's R@10
- **Data scale mattered most**: 5k → 113k images, +15.5 points R@10
- **L2-normalising** VGG features was essential (2% → 49% in a pilot)

Future work

- Train longer, with **10-crop** features
- **Hardest-negative** loss (VSE++), **attention** over image regions
- Compare against the **order-embedding** objective of Vendrov et al.

Thank you

Questions?

`github.com/Vishanth21/neural_caption-image_retrieval`